Inductive step ($t \to t-1$): Assume $\tilde{q}^{\tilde{\pi}}(\tilde{s}', a') = q^{\pi}(f(\tilde{s}'), a')$ for the next state $\tilde{s}'$ and next action a'.

$$
\begin{aligned}
\tilde{q}^{\tilde{\pi}}(\tilde{s}, a) \;&=\; \mathrm{E}_{\tilde{\pi}}\left[\tilde{G}_t \mid \tilde{s}_t = \tilde{s}, A_t = a\right] \;=\; \sum_{\tilde{s}', r} \tilde{p}(\tilde{s}', r \mid \tilde{s}, a) \left[r \;+\; \sum_{a'} \tilde{\pi}(a' \mid \tilde{s}')\, \tilde{q}^{\tilde{\pi}}(\tilde{s}', a')\right] \\
&=\; \sum_{f(\tilde{s}'), g(\tilde{s}'), r} \tilde{p}(\tilde{s}', r \mid \tilde{s}, a) \left[r \;+\; \sum_{a'} \tilde{\pi}(a' \mid \tilde{s}')\, \tilde{q}^{\tilde{\pi}}(\tilde{s}', a')\right] \qquad \text{(A316)} \\
&=\; \sum_{f(\tilde{s}'), G(r, \tilde{s}, a), r} \tilde{p}(\tilde{s}', r \mid \tilde{s}, a) \left[r \;+\; \sum_{a'} \tilde{\pi}(a' \mid \tilde{s}')\, \tilde{q}^{\tilde{\pi}}(\tilde{s}', a')\right] \\
&=\; \sum_{f(\tilde{s}'), r} \tilde{p}(\tilde{s}', r \mid \tilde{s}, a) \left[r \;+\; \sum_{a'} \tilde{\pi}(a' \mid \tilde{s}')\, \tilde{q}^{\tilde{\pi}}(\tilde{s}', a')\right] \\
&=\; \sum_{f(\tilde{s}'), r} p(f(\tilde{s}'), r \mid f(\tilde{s}), a) \left[r \;+\; \sum_{a'} \pi(a' \mid f(\tilde{s}'))\, \tilde{q}^{\tilde{\pi}}(\tilde{s}', a')\right] \\
&=\; \sum_{f(\tilde{s}'), r} p(f(\tilde{s}'), r \mid f(\tilde{s}), a) \left[r \;+\; \sum_{a'} \pi(a' \mid f(\tilde{s}'))\, q^{\pi}(f(\tilde{s}'), a')\right] \\
&=\; q^{\pi}(f(\tilde{s}), a)\;.
\end{aligned}
$$

For the induction step $1 \to 0$ we use $\tilde{p}_0(\tilde{s}_0, r) = p_0(f(\tilde{s}_0), r)$ instead of $\tilde{p}(\tilde{s}', r \mid \tilde{s}, a) = p(f(\tilde{s}'), r \mid f(\tilde{s}), a)$.
It follows that $\tilde{q}^*(\tilde{s}, a) = q^*(f(\tilde{s}), a)$, and therefore

$$
\tilde{\pi}^*(\tilde{s}) \;=\; \operatorname*{argmax}_a \;\; \tilde{q}^*(\tilde{s}, a) \;=\; \operatorname*{argmax}_a \;\; q^*(f(\tilde{s}), a) \;=\; \pi^*(f(\tilde{s}))\;. \qquad \text{(A317)}
$$

Using Bellman's optimality equation would give the same result, where in above equation both $\sum_{a'} \pi(a' \mid f(\tilde{s}'))$ and $\sum_{a'} \tilde{\pi}(a' \mid \tilde{s}')$ are replaced by $\max_{a'}$. □

Theorem A14. *If a Markov decision process $\tilde{\mathcal{P}}$ is state-enriched compared to the MDP $\mathcal{P}$, then for each optimal policy $\tilde{\pi}^*$ of $\tilde{\mathcal{P}}$ there exists an equivalent optimal policy π^* of $\mathcal{P}$, and vice versa, with $\tilde{\pi}^*(f(s)) = \pi^*(s)$. The optimal return is the same for $\tilde{\mathcal{P}}$ and $\mathcal{P}$.*

Proof. The MDP $\tilde{\mathcal{P}}$ is a homomorphic image of $\mathcal{P}$. For state-enrichment, the mapping g is bijective, therefore the optimal policies in $\tilde{\mathcal{P}}$ and $\mathcal{P}$ are equal according to Lemma A1. The optimal return is also equal since it does not change via state-enrichment. □

A7.4 Variance of the Weighted Sum of a Multinomial Distribution

State transitions are multinomial distributions and the future expected reward is a weighted sum of multinomial distributions. Therefore, we are interested in the variance of the weighted sum of a multinomial distribution. Since we have

$$
\mathrm{E}_{s', a'}\left[q^{\pi}(s', a') \mid s, a\right] = \sum_{s'} p(s' \mid s, a) \sum_{a'} \pi(a' \mid s')\, q^{\pi}(s', a')\;, \qquad \text{(A318)}
$$

the variance of $\mathrm{E}_{s', a'}\left[q^{\pi}(s', a')\right]$ is determined by the variance of the multinomial distribution $p(s' \mid s, a)$. In the following we derive the variance of the estimation of a linear combination of variables of a multinomial distribution like $\sum_{s'} p(s' \mid s, a) f(s')$.

A multinomial distribution with parameters $(p_1, \ldots, p_N)$ as event probabilities satisfying $\sum_{i=1}^{N} p_i = 1$ and support $x_i \in \{0, \ldots, n\}$, $i \in \{1, \ldots, N\}$ for n trials, that is $\sum x_i = n$, has

$$
\begin{aligned}
&\text{pdf} && \frac{n!}{x_1! \cdots x_k!}\, p_1^{x_1} \cdots p_k^{x_k}\;, && \text{(A319)} \\
&\text{mean} && \mathrm{E}[X_i] \;=\; n\, p_i\;, && \text{(A320)} \\
&\text{variance} && \mathrm{Var}[X_i] \;=\; n\, p_i\, (1 - p_i)\;, && \text{(A321)} \\
&\text{covariance} && \mathrm{Cov}[X_i, X_j] \;=\; -\, n\, p_i\, p_j\;, \;\; (i \neq j)\;, && \text{(A322)}
\end{aligned}
$$

where X_i is the random variable and x_i the actual count.
A linear combination of random variables has variance

$$\begin{aligned} \text{Var}\left[\sum_{i=1}^{N} a_i\, X_i\right] &= \sum_{i,j=1}^{N} a_i\, a_j\, \text{Cov}\left[X_i, X_j\right] \\ &= \sum_{i=1}^{N} a_i^2\, \text{Var}\left[X_i\right] \,+\, \sum_{i \neq j} a_i\, a_j\, \text{Cov}\left[X_i, X_j\right] \,. \end{aligned} \tag{A323}$$

The variance of estimating the mean X of independent random variables $(X_1, \ldots, X_n)$ that all have variance σ^2 is:

$$\begin{aligned} \text{Var}\left[X\right] &= \text{Var}\left[\frac{1}{n}\sum_{i=1}^{n} X_i\right] \\ &= \frac{1}{n^2}\sum_{i=1}^{n} \text{Var}\left[X_i\right] \,=\, \frac{1}{n^2}\sum_{i=1}^{n} \sigma^2 \,=\, \frac{\sigma^2}{n} \,. \end{aligned} \tag{A324}$$

When estimating the mean $\bar{y}$ over n samples of a linear combination of variables of a multinomial distribution $y = \sum_{i=1}^{N} a_i X_i$, where each y has n_y trials, we obtain:

$$\begin{aligned} \text{Var}\left[\bar{y}\right] &= \frac{\sigma_y^2}{n} \,=\, \frac{1}{n}\left(\sum_{i=1}^{N} a_i^2\, n_y\, p_i\, (1-p_i) \,-\, \sum_{i \neq j} a_i\, a_j\, n_y\, p_i\, p_j\right) \\ &= \frac{n_y}{n}\left(\sum_{i=1}^{N} a_i^2\, p_i\, (1-p_i) \,-\, \sum_{i \neq j} a_i\, a_j\, p_i\, p_j\right) \\ &= \frac{n_y}{n}\left(\sum_{i=1}^{N} a_i^2\, p_i \,-\, \sum_{(i,j)=(1,1)}^{(N,N)} a_i\, a_j\, p_i\, p_j\right) \\ &= \frac{n_y}{n}\left(\sum_{i=1}^{N} a_i^2\, p_i \,-\, \left(\sum_{i=1}^{N} a_i\, p_i\right)^2\right) \,. \end{aligned} \tag{A325}$$

A8 Long Short-Term Memory (LSTM)

A8.1 LSTM Introduction

Recently, *Long Short-Term Memory* (LSTM; [49, 54, 55]) networks have emerged as the best-performing technique in speech and language processing. LSTM networks have been overwhelming successful in different speech and language applications, including handwriting recognition [37], generation of writings [38], language modeling and identification [35, 146], automatic language translation [126], speech recognition [107, 29] analysis of audio data [78], analysis, annotation, and description of video data [22, 136, 123]. LSTM has facilitated recent benchmark records in TIMIT phoneme recognition (Google), optical character recognition, text-to-speech synthesis (Microsoft), language identification (Google), large vocabulary speech recognition (Google), English-to-French translation (Google), audio onset detection, social signal classification, image caption generation (Google), video-to-text description, end-to-end speech recognition (Baidu), and semantic representations. In the proceedings of the flagship conference *ICASSP 2015* (40th IEEE International Conference on Acoustics, Speech and Signal Processing, Brisbane, Australia, April 19–24, 2015), 13 papers had "LSTM" in their title, yet many more contributions described computational approaches that make use of LSTM.
The key idea of LSTM is the use of memory cells that allow for constant error flow during training. Thereby, LSTM avoids the *vanishing gradient problem*, that is, the phenomenon that training errors are decaying when they are back-propagated through time [49, 52]. The vanishing gradient problem severely impedes *credit assignment* in recurrent neural networks, i.e. the correct identification of relevant events whose effects are not immediate, but observed with possibly long delays. LSTM, by its constant error flow, avoids vanishing gradients and, hence, allows for *uniform credit assignment*,

i.e. all input signals obtain a similar error signal. Other recurrent neural networks are not able to assign the same credit to all input signals, therefore they are very limited concerning the solutions they will find. Uniform credit assignment enabled LSTM networks to excel in speech and language tasks: if a sentence is analyzed, then the first word can be as important as the last word. Via uniform credit assignment, LSTM networks regard all words of a sentence equally. Uniform credit assignment enables to consider all input information at each phase of learning, no matter where it is located in the input sequence. Therefore, uniform credit assignment reveals many more solutions to the learning algorithm which would otherwise remain hidden.

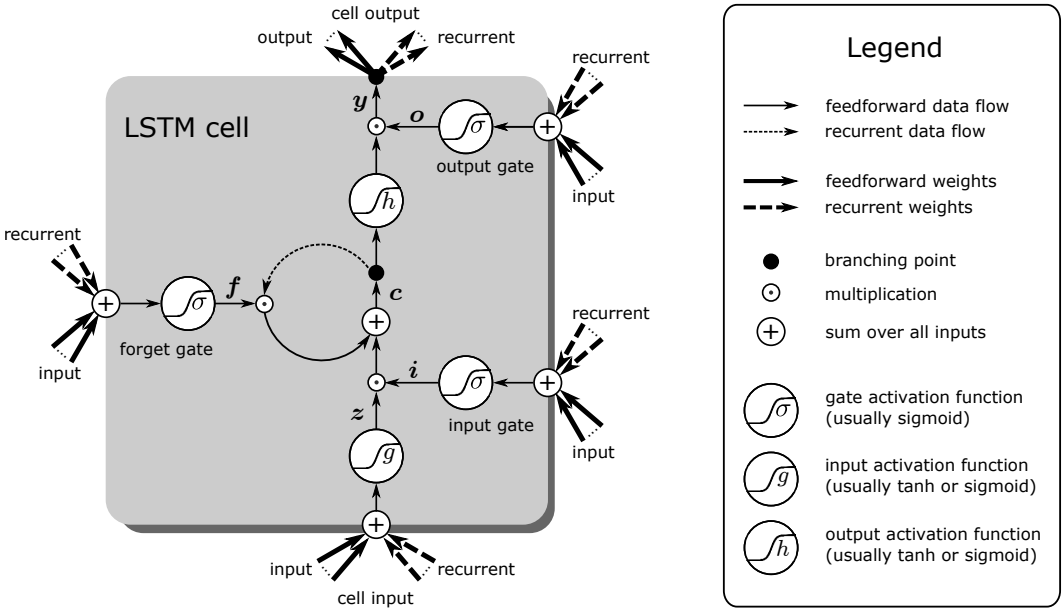

Figure A10: LSTM memory cell without peepholes. $\boldsymbol{z}$ is the vector of cell input activations, $\boldsymbol{i}$ is the vector of input gate activations, $\boldsymbol{f}$ is the vector of forget gate activations, $\boldsymbol{c}$ is the vector of memory cell states, $\boldsymbol{o}$ is the vector of output gate activations, and $\boldsymbol{y}$ is the vector of cell output activations. The activation functions are g for the cell input, h for the cell state, and σ for the gates. Data flow is either "feed-forward" without delay or "recurrent" with an one-step delay. "Input" connections are from the external input to the LSTM network, while "recurrent" connections take inputs from other memory cells and hidden units of the LSTM network with a delay of one time step.

A8.2 LSTM in a Nutshell

The central processing and storage unit for LSTM recurrent networks is the *memory cell*. As already mentioned, it avoids vanishing gradients and allows for uniform credit assignment. The most commonly used LSTM memory cell architecture in the literature [39, 112] contains forget gates [31, 32] and peephole connections [30]. In our previous work [57, 53], we found that peephole connections are only useful for modeling time series, but not for language, meta-learning, or biological sequences. That peephole connections can be removed without performance decrease, was recently confirmed in a large assessment, where different LSTM architectures have been tested [40]. While LSTM networks are highly successful in various applications, the central memory cell architecture was not modified since 2000 [112]. A memory cell architecture without peepholes is depicted in Figure A10.

In our definition of a LSTM network, all units of one kind are pooled to a vector: $\boldsymbol{z}$ is the vector of cell input activations, $\boldsymbol{i}$ is the vector of input gate activations, $\boldsymbol{f}$ is the vector of forget gate activations, $\boldsymbol{c}$ is the vector of memory cell states, $\boldsymbol{o}$ is the vector of output gate activations, and $\boldsymbol{y}$ is the vector of cell output activations. We assume to have an input sequence, where the input vector at time t is $\boldsymbol{x}^t$. The matrices $\boldsymbol{W_z}$, $\boldsymbol{W_i}$, $\boldsymbol{W_f}$, and $\boldsymbol{W_o}$ correspond to the weights of the connections between inputs and cell input, input gate, forget gate, and output gate, respectively. The vectors $\boldsymbol{b_z}$, $\boldsymbol{b_i}$, $\boldsymbol{b_f}$, and $\boldsymbol{b_o}$ are the bias vectors of cell input, input gate, forget gate, and output gate, respectively. The activation functions are g for the cell input, h for the cell state, and σ for the gates, where these functions are evaluated in a component-wise manner if they are applied to vectors. Typically, either the sigmoid